



XDNA ISA and GEMM Kernels

Tamino Steinert

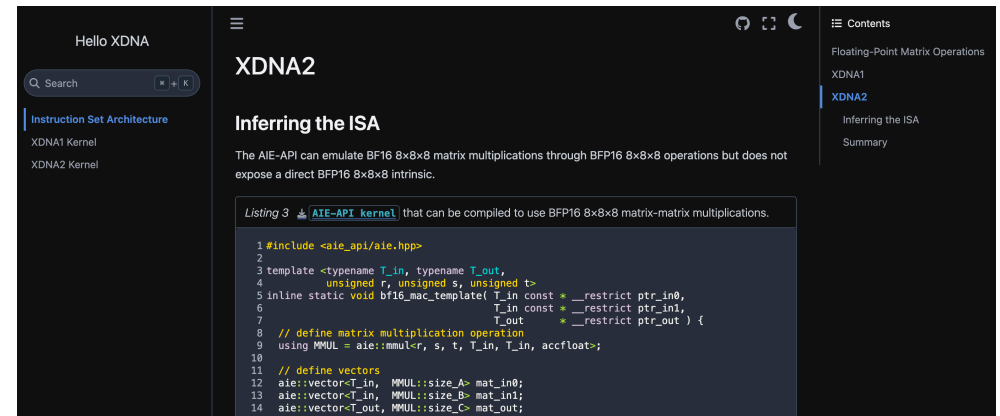
XDNA ISA

XDNA ISA Overview

- VLIW ISA with 6 instruction slots; one for each functional unit.
- Instruction slots:
 - V** — Vector Unit
 - A** — Load Unit A
 - B** — Load Unit B
 - S** — Store Unit
 - X** — Scalar/Control Unit
 - M** — Movement Unit
 - XM** — Scalar/Control Unit and Movement Unit
- **XM** slot can have 32-bit immediate; occupies **X** and **M** slots.
- Register Files:
 - DM0–DM4** — 2048-bit accumulator registers
 - Y0–Y5** — 1024-bit vector registers
 - R0–R31** — 32-bit scalar registers
 - P0–P7** — 20-bit pointer registers
 - M0–M7** — 20-bit modifier registers

```
1 // Three different NOPs for whole processor
2 nopv; nopa; nopb; nops; nopx; nopm // 16 bytes
3 nopv; nopa; nopb; nops; nopxm      // 16 bytes
4 nop                                // 2 bytes
```

```
1 // Move operations and their immediate sizes
2 movx r0, #128 // 2^7; 8-bit imm.
3 mova r0, #128 // 2^7; 8-bit imm.
4 mov r0, #512 // 2^9; 10-bit imm.
5 movxm r0, #2147483648 // 2^31; 32-bit imm.
```



Hello XDNA — XDNA2

Register Files

AIE-ML v2

AMD

Technical Information Portal

Sign In

← Search

Versal Adaptive SoC AIE-ML v2 Architecture Manual (AM027)

AM027

2025-10-23

1.1 English

🔍

🔖

📄

⋮

Register Files

🔖 🗨️

The AIE-ML v2 has several types of registers. Some of the registers are used in different functional units. This section describes the various types of registers.

Scalar Registers

🔖 🗨️

Scalar registers include configuration registers. See the following table for register descriptions.

Table: Scalar Registers

Syntax	Number of bits	Description
r0..r31	32 bits	General-purpose registers
m0..m7	20 bits	Modifier registers
p0..p7	20 bits	Pointer registers

Special Registers

🔖 🗨️

Table: Special Registers

Syntax	Number of bits	Description
dn0..dn7	20 bits	AGU dimension size register
dj0..dj7	20 bits	AGU dimension stride (jump) register
dc0..dc7	20 bits	AGU dimension count register
s0..s3	6 bits	Shift control
sp	20 bits	Stack pointer

XDNA2

☰

🔍

🌙

🔍

☰

Summary

XDNA2 has five 2048-bit accumulator registers (DM0 – DM4) and their 1024-bit/512-bit views in the ISA. Compared to AIE-ML v2, this represents a 1.6x reduction in the architecturally visible accumulator count.

Table 3, Table 4, Table 5, and Table 6 summarize additional information on registers that are relevant for BFP16 operations. The inferred ISA is provided in Table 7.

Table 3 Exponent Registers.

32-bit	64-bit
EL0	E0
EH0	
...	...
...	
EL11	E11
EH11	

Table 4 Vector Registers with Exponent Registers.

8-byte + 64-byte	72-byte
E0 + X0	EX0
...	...
E11 + X11	EX11

Vector Slot — V

- Registers:
 - Destination: accumulator registers
 - Source: accumulator and vector registers
 - Configuration: scalar registers

```
1 // fp32 64 element-wise addition
2 mova r0, #60
3 vadd.f dm0, dm0, dm1, r0
```

```
1 // bf16 64 element-wise multiplication
2 mova r0, #60
3 vmul.f dm1, y3, y0, r0
```

```
1 // bfp16ebs8 8x8x8 matrix multiplication
2 mova r0, #780
3 vmac.f dm0, dm0, ex0, ex2, r0
```

AI Engine API User Guide (AIE-API) 2025.2

[Overview](#)
[Related Pages](#)
[API Reference](#)

AI Engine API User Guide

- Overview
- Changelog
- Deprecated List
- API Reference
 - Basic Types
 - Configuration
 - Memory
 - Initialization
 - Arithmetic
 - Bits
 - Comparison
 - Reduction
 - Reshaping
 - Floating-point Conversion
 - Elementary Functions
 - Matrix Multiplication
 - aiemmul< M_Elems, K_Elems, N_Elems>
 - Matrix Multiplication Modes**
 - Supported Matrix Multiplication
 - GEMM leveraging multidimensional
 - Supported Sparse Matrix Multiplication
 - Fast Fourier Transform (FFT)
 - Special Multiplications
 - Lookup Tables
 - Operator Overloading
 - Interoperability with Adaptive Data Format
 - Utility functions and classes

Supported Matrix Multiplication Modes

Note
In the following table, the following abbreviations are used to denote low precision floating point multiplications:

- f8 is used to denote multiplication with 8 bit low precision floating point formats (bfloat8 x bfloat8, bfloat8 x float8, float8 x bfloat8, float8 x float8).
- f16 is used to denote multiplication with 16 bit floating point formats (bfloat16 x bfloat16, bfloat16 x float16, float16 x bfloat16, float16 x float16). If a multiplication mode is restricted to a specific combination, then the unabbreviated format name is used.

Matrix multiplication modes for re										
Arch.	8b x 4b	8b x 8b	16b x 8b	8b x 16b	16b x 16b	32b x 16b	16b x 32b	32b x 32b ^c	f8 x f8	bfloat16 x bfloat16
AIE		4x8x4 4x16x4 ^a 8x8x4 ^a 2x8x8 4x8x8 ^a 1x16x8	4x4x4 8x4x4 ^a 4x8x4 ^a 4x4x8 ^a	4x4x4 ^a 4x4x4 ^a 8x8x1 ^{ab}	4x4x4 ^a 2x4x8 ^a 4x4x8 ^a 8x8x1 ^{ab}	2x4x8 ^a 4x4x4 ^a 2x2x4 2x4x4 ^a 4x4x2 ^a 2x2x8 ^a	4x2x2 2x4x8 ^a 4x4x4 ^a 2x4x2 ^a 4x4x2 ^a 2x4x4 ^a 4x4x1 ^a	4x2x4 ^a 2x2x2 2x4x2 ^a 2x8x2 ^a 4x4x2 ^a 2x4x4 ^a 4x4x1 ^a		
AIE-ML/XDNA1	4x16x8 8x16x8 ^a 4x32x8 ^{ab}	4x8x4 ^{ab} 4x16x4 ^{ab} 8x8x4 ^{ab} 2x8x8 4x8x8 8x8x8 ^a 1x16x8 ^{ab} 2x16x8 ^{ab} 4x16x8 ^{ab}	4x4x4 ^{ab} 8x4x4 ^{ab} 4x8x4 4x4x8 8x4x8 ^{ab} 2x8x8	4x4x8 ^{ab} 4x4x4 ^{ab}	4x4x4 2x4x8 4x4x8 ^{ab} 4x2x8 8x2x8 ^a 8x1x8 ^{ab}	2x4x8 4x4x8 ^{ab} 4x4x4 4x2x4 4x1x8 ^{ab}	2x4x8 4x4x4 8x2x4 ^a 4x1x8 ^{ab} 8x1x8 ^{ab}	4x2x4 ^a 4x4x4 ^{ab} 8x2x4 ^a 4x1x8 ^{ab} 8x1x8 ^{ab}		4x8 8x8 4x1 8x8
XDNA2	4x16x16	4x8x8 8x8x8	4x4x8 8x4x8 4x8x8 2x8x8 ^b	8x2x8 ^b 4x4x8 ^b	4x2x8 ³² 8x2x8 ³² 2x4x8 ⁶⁴ 4x4x8 ⁶⁴ 8x1x8 ^{b, 32}	4x2x8 2x4x8 ^{ab} 4x4x8 ^{ab} 4x1x8 ^b	4x4x8 ^{ab}	4x2x8 ^{ab} 4x4x4 ^{ab} 4x4x8 ^{ab} 8x2x8 ^{ab} 4x1x8 ^b		8x8 4x8 4x8 8x8 8x1
AIE-MLV2	4x16x16 ^{ab} 8x8x8	4x8x8 8x8x8	8x2x8 ^b 8x4x8 ^{ab} 4x4x8 ^b 4x8x8 ^{ab}	8x2x8 ^b 4x4x8 ^b	4x2x8 ^{b, 32} 8x2x8 ³² 2x4x8 ⁶⁴ 4x4x8 ⁶⁴ 8x1x8 ^{b, 32}	4x2x8 2x4x8 ^{ab} 4x4x8 ^{ab} 4x1x8 ^b	4x4x8 ^{ab}	4x2x8 ^{ab} 4x4x4 ^{ab} 4x4x8 ^{ab} 8x2x8 ^{ab} 4x1x8 ^b	8x8x8	4x8 4x8 8x8 8x1

AMD — Supported Matrix Multiplication Modes — AIE-API

Load/Store Slots

Load Unit A — A

- Registers:
 - Destination: scalar, vector or accumulator registers
 - Source: L1 scratchpad + pointer register

Load Unit B — B

- Registers:
 - Destination: scalar or vector registers
 - Source: L1 scratchpad + pointer register

- All units have an address generation unit (AGU)

Store Unit — S

- Registers:
 - Destination: L1 scratchpad
 - Source: scalar, vector or accumulator registers + pointer register

```
1 vlda bml0, [p0, #0];  
2 vldb x0, [p1, #0];  
3 vst bml0, [p2, #0]
```

```
1 vlda x1, [p0, #64];  
2 vldb r0, [p1, #128];  
3 vst.conv.bf16.fp32, cml0 [p2, #128]
```

```
1 vlda.conv.fp32.bf16 cmh2, [p0], #256;  
2 vldb wl2, [p5], #-128;  
3 vst, cml0 [p2], #256
```

Scalar/Control Slot — X

Scalar operations

- Registers:
 - Destination: scalar registers
 - Source: scalar registers or immediate

```
1 movx r4, #67
```

```
1 add r0, r1, r2
```

```
1 sub r3, r2, #42
```

```
1 mul r4, r2, r7
```

```
1 ltu r16, r12, r30
```

```
1 sel.nez r28, r4, r7, r27
```

Control operations

- Registers:
 - Destination: PC
 - Source: scalar registers or immediate
- Control operations have a 6-cycle latency, i.e., no other control operation can be scheduled within those cycles.
- Destination address must be 16-byte aligned.

```
1 .p2align 4
2 .my_label:
3 // ...
4 jnz r0, #.my_label
```

```
1 ret lr
```

Move Slot — M

- Registers:
 - Destination: all
 - Source: all
 - Configuration: scalar registers

```
1 // Move 32 bits
2 mov r0, r1
```

```
1 // Move last 20 bits from r6 to m0
2 mov m0, r6
```

```
1 // Move 512 bits
2 vmov bml14, x4
```

```
1 // Convert from fp32 to bfp16ebs8
2 vconv.bfp16ebs8.fp32 ex0, dm0
```

```
1 // Shuffle ???
2 mova r0, #29
3 vshuffle x0, x0, x0, r0
```

AMD AI Engine-ML Intrinsic User Guide (v2025.2)

Overview API Reference Search

AI Engine-ML Intrinsic User Guide
Datatype and intrinsic documentation
API Reference
Accumulator Data Types
Compiler optimizations
Load/Store Operations
Scalar Data Types
Scalar Operations
Stream access
Vector Conversions
Vector Data Types
Vector Operations
Add/Subtract
Bitwise logical
Compare/Select
Initialization
Multiply Accumulate
Shift
Shift element
Shuffle
Illustration of Shuffle Modes
eShuffleMode
shuffle
shuffle
shuffle
shuffle
shuffle
shuffle
shuffle
shuffle
shuffle
shuffle
shuffle

Illustration of Shuffle Modes

Vector Operations » Shuffle

The following images illustrate the behavior of the shuffle modes. Each output is identified by a mode label. Every circle in the figure corresponds to one byte. The input is shown on the opposite side. The two input registers are concatenated and the output is produced depending on the selected mode. Different modes are used to access the low or high 512 bits of the output. Some modes do not produce more matrices that require more than 512 bit. In those cases the second input register is not accessed.

A special mode is `shuffle_T16_1x2_flip`. This one does not transpose the matrix, instead it flips the inner dimension. This means that the order of values along the dimension is reversed. Since the inner dimension of this mode has exactly two values of 16 bits, this is the same as swapping each neighbouring 16 bit values.

AMD — Illustration of Shuffle Modes — AIE-ML

GEMM Kernels

Datatypes

FP32

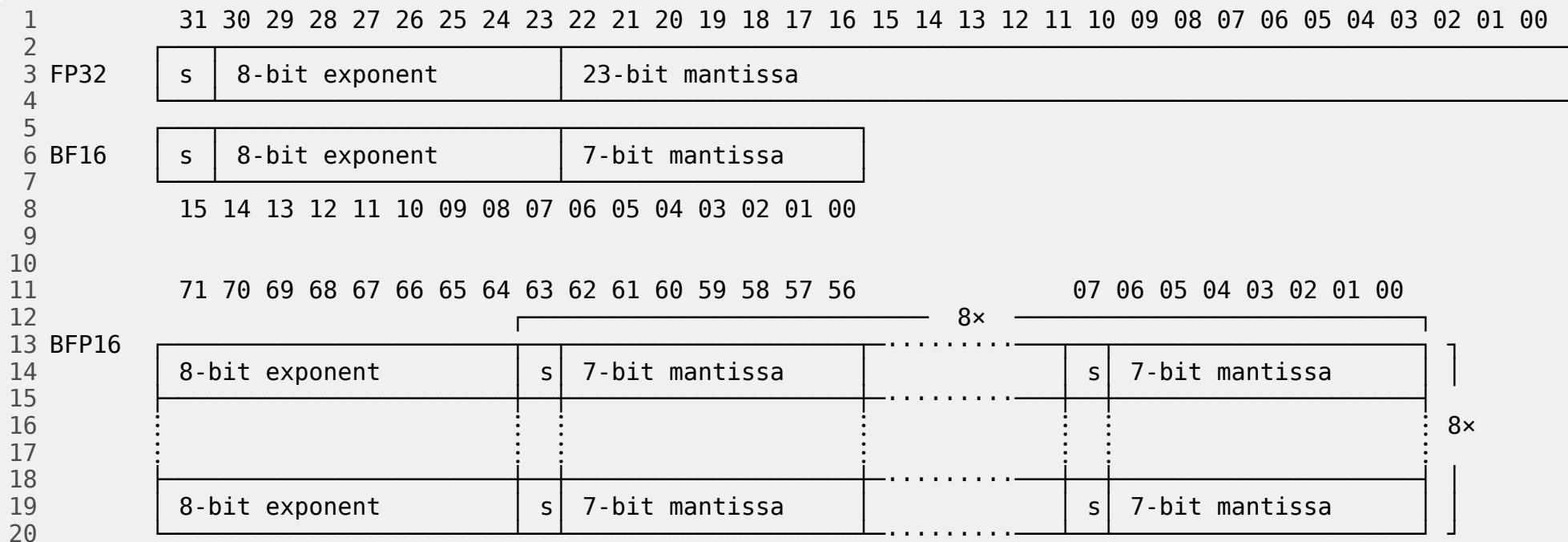
- 32 bits per element
- Hidden bit

BF16

- 16 bits per element
- Hidden bit

BFP16 (**bfp16ebs8**)

- 9 bits per element
- No hidden bit



GEMM Overview

- **Goal:** Perform matrix multiplication with **8x8x8-bfp16xbfp16** operations.
- **Steps:**
 1. Load BF16 output and convert to FP32
 2. Load BF16 inputs and convert to BFP16
 3. Multiply BFP16 inputs and accumulate to FP32 output
 4. Store FP32 output as BF16
- **Notes:**
 - **8x8x8-bfp16xbfp16** performs **mk, nk -> mn** with **m, n, k=8**
 - **8x8-bfp16** has 576 bits and is stored in **ex[0-11]** (exponents are stored in **e[0-11]**)

- **Useful operations:**

- Load BF16 data into vector register:

```
1 vlda x0, [p0], #0
2 vl原因db x0, [p0], #0
```

- Load BF16 data into accumulator register (FP32):

```
1 vlda.conv.fp32.bfp16 cmh2, [p0], #0
```

- Convert FP32 data to BFP16:

```
1 vconv.bfp16ebs8.fp32 ex0, dm0
```

- Create a vector register with ones:

```
1 movxm r0, #16256 // bfp16-one as int
2 vbcst.16 x0, r0 // broadcast register
3 vmov x1, x0 // x0 contains 1s
```

- Transpose a vector (**kn->nk**, **y0=kn**, **y1=nk**):

```
1 vshuffle x2, x0, x1, r0 // r0 = 52
2 vshuffle x3, x0, x1, r1 // r1 = 53
```

- Element-wise multiplication (**BF16** → **FP32**)

```
1 vmul.f dm0, y1, y0, r0 // r0 = 60
```

- Store acc. register (FP32) and convert to BF16:

```
1 vst.conv.bfp16.fp32 cml0, [p2, #0]
```

How to Optimize a GEMM Kernel for the XDNA Processor

- **Goal:** Perform a `vmac.f` operation *every* cycle.
- **Theoretical peak performance:**
 $8 \times 8 \times 8 \times 2 \text{ FLOPs} \times 1.8 \text{ GHz} = 1843.2 \text{ GFLOPS}$
- **Strategies:**
 - Use most efficient operations for compute, loading and storing.
 - Maximize register reuse.
 - Schedule operation as early as possible.
 - Overlap load/store operations with compute operations.
 - Hide branch latencies behind other operations.
- **Questions to answer:**
 - What are the most efficient operations?
 - What register blocking is the best performing?
 - What latency does each operation have?
 - How to schedule load/store operations on units?
 - When is the branching condition known?